

A Modeling Method for Conceptualizing and Aligning Software Architectures

Sven Christ

Preface

XXX

Contents

1	Introduction	5
2	Methodology	8
	Bibliography	16

Preliminary Philosophical Considerations

A peculiar intellectual roadblock halted my initial attempt to conduct this research before it began. The fundamental question I faced concerned the nature of research itself. How could I proceed to conduct research without first explicating what research precisely is? Upon inquiry, I came to regard research as the process of attaining the aim of any scientific endeavor: *expanding the body of human knowledge*.

This definition, however, generated a vexing new question and led me down the rabbit hole of epistemology: *What is knowledge?* Linguistically, this question constitutes a sensical inquiry. To answer it – or any question – I would need to *know* the answer, thus employing knowledge to define the concept of knowledge, which is evidently nonsensical.

The reader might possess a peripheral awareness of the problem I faced and perhaps has conquered this obstacle in their own work in some way or another. I, however, could not proceed without addressing this foundational issue.

I acknowledge, I can – in the format of this text – use mere words to communicate my thoughts. My first request, thus, is for you to conspire with me and accept that what I will attempt to say, can, in fact, be verbalized. Otherwise, how may the nature of knowledge be examined when the words employed to that end are required to be true and thus contain knowledge themselves?

Note: I consciously decided on the term *word*, to differentiate it from *language* and other symbols. See, also, the “Triangle of Reference” (Ogden et al. 1923).

* * *

To begin, let us face a meta-ontological question and ask how we can define (in words) any concept (including knowledge) that we can grasp with our mind. Axiomatically, we must accept that our cognition will map the concepts in our mind *appropriately* onto the words we say. This statement remains a necessary assumption for any well-meaning discussion. Any attempt to investigate what constitutes this appropriateness (i. e. circumstances, time, individualities) is futile, because the spoken words are the only gateway to the mind of the speaker (Wittgenstein 1953). Any attempt to skip the words and move our focus towards our own mind would inevitably lead to methodological solipsism (Putnam

1975, p. 136), which would render any communication about the concepts contained within the mind redundant.

Furthermore, evaluating the pure truth of linguistic statements is impossible due to the infinite regress inherent in defining words. Every word used to express a statement relies on a definition composed of other words, which themselves require further definitions. This creates an endless chain where each definition depends on another, extending ad infinitum. As a result, attempting to fully comprehend a statement through its linguistic components leads us into an infinite loop of definitions without ever arriving at a fundamental understanding. This infinite regress prevents us from establishing the absolute truth of any statement based solely on language, as the foundational meanings necessary for such an evaluation are perpetually out of reach. This underscores the inherent limitations of language as a medium for conveying pure, unequivocal truth.

It follows, then, that words can only obtain *meaning* when they – not their truth, but rather their sense – can be validated against something outside the mind of the speaker. That is, when no one hears me, speaking itself becomes nonsensical (Wittgenstein et al. 1933). I can call the sky “blue” today, and “green” tomorrow and “red” the day after. And each statement would be true and would have to pass as knowledge, because there is no way (and no need) to proof it. Everything I would say under these circumstances becomes true by default.

Let us, thus, postulate that knowledge can only exist *intersubjectively*. Which means, that the words I use to verbalize my mind’s concepts must correspond – when listened to – to *equivalent* concepts in the mind of the listener. How can we be sure that our words correspond precisely to equivalent concepts in the minds of others?

We cannot, as others’ minds are inaccessible to us. This leads us to consider the role of *communal verification* in the establishment of knowledge. Statements may become true through social endorsement and collective agreement.

Is this an arbitrary agreement, though, or is there a deeper, more fundamental basis for shared meaning?

* * *

Even with no other human listening to us, we can never speak into the void. Nature will always be there. We can not lie to nature nor escape it. However, nature does not speak any natural language. Thus, we can not ask it to explain itself. We may only ever ask: yes or no? And we do that by executing actions, derived from our beliefs and expressed in natural language, against nature and investigating the resonance. Over time we can build theories about nature and express them in words under the assumption that

other humans interpreting these words have the same understanding and use the same words to describe the phenomena witnessed in nature (Aristotle 1933, 1011b). This is the basis for any research in the natural sciences. Because the static and tangible subject of research allows us to build theories and retreat to the methods of fallibilism. The natural sciences escape any definition of knowledge by this. Instead, building their argument on top of pragmatism: Truth is a means to an end to make decisions. Can we use the same methodological approach and the same epistemological paradigms when considering a contingent research subject like information systems and the human mind (Frank 2007)? See, “Correspondence Theory of Truth” (David 2022).

Again, how can we be sure that the words we speak will evoke the same mental concepts in the listener as in ourselves? To repel any ad absurdum discussions about the “nature of reality”, let us follow a phenomenological approach to this question (Husserl 2002). See, “Twin Earth thought experiment” (Putnam 1973).

One approach is to, again, consider words to be our portal into the human mind (see above) and enter a *dialogue* with other humans about what constitutes knowledge. But which words – which knowledge – will ultimately be accepted? We will accept that, which is most useful and effective to facilitate successful actions in practical contexts. This means, that truth – among humans – is determined by the practical outcomes of holding certain beliefs. See, the “Pragmatic Theory of Truth” (Capps 2023).

This leads me to postulate that knowledge exists *sui generis*, in a class of its own. Knowledge is not contingent upon explanation, but on its inherent capacity to produce practical outcomes. Truth is, what is practical, what is useful, what is effective. We do not need to explicitly break knowledge down into further components like justification and belief. See, “Knowledge and its Limits” (Williamson 2002).

But what is a “practical outcome”? How can we evaluate the pragmatism of something when we have no mirror – like nature – to hold up against our beliefs and actions.

* * *

Evaluating the pragmatism of something necessitates an acknowledgment of different *perspectives*. Because the individual and its context dictate how word configurations – potential truths – are understood and interpreted. What is useful for someone may not be useful for another. But how then, could we find universal agreement on the pragmatism of something?

I see 3 possible resolutions. First, consider the individual to be the source for evaluating pragmatism. This would lead to a solipsistic approach, which I have already refuted. Second, consider the collective to be the source for evaluating pragmatism. This would

lead to a democratic approach, which has little to do with the intellectual pursuit of knowledge; too much loss and friction. Third, consider as pragmatic that which enhances the potential for usefulness. That, which increases our understanding of the world and offers insights but does not prescribe them.

* * *

I wanted to define what knowledge is. I have first followed a philosophy of language approach and asked how we can define, using words, *anything*. We have seen that shared meaning is a requirement.

How can we proof that we actually mean the same thing by the words we say? I have looked at the natural sciences and stated that it is the pragmatic content that makes knowledge knowledge. This means, knowledge is not something that statements possess but rather that knowledge is the thing that has the most benefit toward and end. This also means that we can never know anything for certain. We can just know enough for a specific goal.

To advance these thoughts one step further and answer what pragmatism is and how it can be evaluated, I decided on the expansion of the linguistically accessible universe.

The ultimate contribution of this dissertation is to enhance our ability to bring about practical outcomes in information systems by expanding, resolving and properly arranging the universe of discourse.

1 Introduction

Information systems collect, process, store, and distribute data to provide information and knowledge. Today, information systems integrate hardware, software, data, processes, and people to support business operations and management. Holistic modeling of information systems (ArchiMate, ...) is still a growing body of research. Likewise, the modeling of processes (BPMN, ...) and the modeling of data (ERM, ...) are well-established fields. Yet, Little attention has been paid to the modeling of *software* within the context of information systems. Existing software modeling approaches (UML, ...) fall short because they focus on the design of individual software components rather than the architecture of the entire software system.

The analogy of architecture from the physical realm, as traditionally understood in the planning and design of buildings and structures, has long been employed in computer science to conceptualize and communicate the high-level structure of software systems. This metaphor, while useful, introduces a layer of metaphorical complexity due to the intangible and dynamic nature of software as opposed to the tangible and static nature of physical structures. Unlike buildings, software systems are inherently adaptable and mutable, often undergoing continuous evolution to meet changing user needs and technological advancements.

From the perspective of information systems, the architecture of software systems holds particular significance. The concept of architecture in this context emphasizes not only the system's structure but also the alignment of the system with processes, people and, ultimately, business objectives. Despite the critical role of software architecture, there exists a gap between theoretical insights and practical applications. Current modeling languages like the Unified Modeling Language (UML) and ArchiMate offer various means to represent software architectures but often fall short in addressing the full spectrum of architectural concerns. UML, for instance, provides detailed component diagrams but may lack in expressing higher-level architectural abstractions effectively. ArchiMate, while useful for high-level enterprise architecture modeling, might operate at an abstraction level too elevated for practical software architecture needs, rendering it expressively weak in certain contexts.

This dissertation emerges at the intersection of these challenges and seeks to contribute to the software architecture literature. The focus is on developing and evaluating a novel modeling method that addresses the challenges of software architecture design, communication, and maintenance in the context of business information systems. The

key objective is to synthesize a reflected, differentiated view on software architecture, building upon a sound empirical and theoretical foundation.

* * *

The motivation for this research is rooted in the high complexity of modern software systems and the limitations of existing modeling approaches in effectively capturing and communicating architectural nuances. By enhancing the cognitive comprehension of software architectures through improved modeling methods, organizations can achieve better alignment between software systems and business objectives, facilitate smoother transitions, and enhance overall productivity.

Moreover, the dynamic nature of software development, characterized by rapid technological changes and evolving user requirements, necessitates architectures that are not only well-designed but also adaptable. The lifecycle of software is accelerating, making it imperative to consider evolutionary aspects within architectural models. This includes strategies for designing architectures that can accommodate future changes without necessitating complete overhauls, thereby preserving investments and maintaining system integrity over time.

Addressing the complexities of software architecture involves grappling with several conceptual challenges:

1. *Multiformity*: Software systems vary widely in form and scale, from small embedded systems to global cloud applications. Crafting a universal definition or modeling approach that encompasses this diversity is inherently challenging.
2. *Separating the Variant from the Invariant*: Distinguishing between foundational architectural concepts and those subject to change is crucial for developing robust models.
3. *Accounting for the Dynamics of Change*: Architectural models must anticipate and facilitate future evolution, requiring an understanding of how present decisions impact adaptability.
4. *Multiple Levels of Abstraction*: Software architecture operates across various levels of abstraction, from high-level designs to detailed specifications. Developing concepts to represent architectures across these levels is essential.
5. *Metaphorical Complexity*: The metaphor of architecture introduces complexities when applied to software due to its intangible and dynamic nature.

To address these challenges and contribute meaningful advancements in the field, this dissertation is guided by several key objectives. Each objective is designed to tackle a specific facet of the overarching challenges.

1. *Achieving Conceptual Clarity about Software Architecture*: Establishing a clear, scientifically sound definition of software architecture that encompasses its multifaceted nature, including both functional and non-functional requirements. This definition should serve as a reference point for future discussions and research.
2. *Differentiated Reconstruction of Prospective Users' Requirements*: Understanding the needs and challenges faced by software architects and developers through empirical studies. This involves capturing practical insights to inform the development of modeling languages and tools that are aligned with real-world practices.
3. *Thorough Reconstruction of Existing Approaches*: Critically examining current modeling languages such as UML and ArchiMate to assess their strengths, weaknesses, and application areas. This analysis is essential to identify gaps and inform the development of new or complementary modeling approaches.
4. *Developing a Dedicated Domain-Specific Modeling Language and Modeling Method*: Creating a modeling language that integrates theoretical insights with practical challenges, capable of representing the complexity of modern software architectures. The language should be intuitive and accessible to promote widespread adoption within the developer community.

The outcomes of this research hold significant potential for making substantial contributions to both the academic community and practical industry applications. By advancing theoretical understanding, this research enriches the foundational knowledge that underpins software architecture as a discipline. It provides new perspectives and frameworks that can stimulate further scholarly inquiry. On the practical side, the research offers tangible tools and methods that can enhance the way software systems are designed and managed, leading to improved efficiency and effectiveness in the industry. By fostering a closer alignment between theoretical concepts and practical needs, the research bridges a critical gap, facilitating collaboration and knowledge transfer between academia and industry.

1. *Theoretical Advancement*: By addressing the conceptual challenges and establishing a clear definition of software architecture, the research contributes to the theoretical foundation of the field.
2. *Practical Impact*: The development of a new modeling method and language can directly impact software development practices, leading to improved communication, design, and maintenance of software systems.
3. *Bridging Theory and Practice*: By aligning theoretical insights with practitioner requirements, the research bridges the gap between academia and industry, ensuring that the outcomes are both scientifically rigorous and practically relevant.

2 Methodology

1. Artifact Excavation (Data Collection)

Objective: Gather a diverse and representative sample of software architectures.

- Source Code Repositories: Collect architectures from open-source projects on platforms like GitHub, GitLab, and Bitbucket.
- Documentation: Obtain architectural diagrams, design documents, and technical specifications from project documentation.
- Interviews and Surveys: Conduct interviews with software architects and developers to gain insights into architectural decisions and rationale.
- Legacy Systems Analysis: Include older software systems to understand historical architectural practices.

Considerations:

- Ethical Compliance: Ensure all collected data respects intellectual property rights and privacy laws.
- Diversity: Aim for a variety of software types (e.g., web applications, mobile apps, embedded systems) and domains (e.g., healthcare, finance, education).

2. Classification and Cataloging

Objective: Organize the collected architectures into meaningful categories.

- Structural Classification:
 - Monolithic vs. Microservices: Distinguish between tightly coupled and loosely coupled systems.
 - Layered vs. Event-Driven: Categorize based on architectural patterns.
- Functional Classification:
 - Domain-Based: Group architectures by industry or application domain.
 - Technology Stack: Classify based on programming languages, frameworks, and platforms used.
- Temporal Classification:
 - Era-Based: Organize architectures by the period they were developed (e.g., mainframe era, client-server era, cloud era).

Tools:

- Databases: Use relational or graph databases to store and query architectural data.
- Taxonomy Software: Employ tools like Protégé for ontology development.

3. Structural Analysis

Objective: Analyze the internal components and their relationships within each architecture.

- **Component Identification:** Break down architectures into their fundamental components (e.g., modules, services, layers).
- **Relationship Mapping:** Diagram the interactions and dependencies between components.
- **Pattern Recognition:** Identify common architectural patterns (e.g., MVC, RESTful services) and anti-patterns.
- **Metric Calculation:**
 - **Complexity Metrics:** Evaluate cyclomatic complexity, coupling, and cohesion.
 - **Scalability and Performance Indicators:** Assess load times, response rates, and throughput.

Methods:

- **Static Code Analysis:** Use tools like SonarQube or Understand.
- **Visualization:** Create UML diagrams or use visualization tools like Gephi.

4. Contextual Analysis

Objective: Understand the architectures within their broader socio-technical contexts.

- **Historical Context:** Investigate the technological landscape and limitations during the time of development.
- **Organizational Factors:** Examine how company culture, team structure, and project management methodologies influenced architectural decisions.
- **Economic and Market Forces:** Consider market demands, competition, and economic constraints.

Data Sources:

- **Project Histories:** Review version histories, commit messages, and project timelines.
- **Industry Reports:** Consult market analyses and technology trend reports.

5. Comparative Analysis

Objective: Compare architectures to identify trends, commonalities, and divergences.

- **Cross-Artifact Comparison:** Analyze similarities and differences across architectures within the same category.
- **Temporal Trends:** Observe how architectures have evolved over different technological eras.
- **Best Practices Identification:** Determine architectural features associated with successful projects.

Approach:

- **Statistical Analysis:** Use statistical methods to find correlations.
- **Case Studies:** Develop in-depth studies of particular architectures.

6. Interpretation and Hypothesis Formation

Objective: Derive insights and formulate definitions based on the analyses.

- **Definition Refinement:** Use findings to define what constitutes a software architecture.
- **Boundary Setting:** Clarify what is not considered part of software architecture (e.g.,

low-level code optimizations). • Theory Development: Propose theories on how and why certain architectural styles emerge.

Validation:

• Peer Review: Present findings to other researchers for feedback. • Expert Consultation: Engage with seasoned software architects for validation.

7. Documentation and Preservation

Objective: Ensure that all findings are thoroughly documented and preserved for future research.

• Archiving: Store all collected data, analyses, and interpretations in accessible repositories. • Standardized Reporting: Use consistent formats for documentation to facilitate sharing and collaboration.

Tools:

• Digital Libraries: Use platforms like Zenodo or Figshare. • Version Control: Employ Git for tracking changes in research documents.

8. Dissemination

Objective: Share the research outcomes with the broader community.

• Publications: Write research papers, articles, and reports. • Conferences and Workshops: Present findings at industry and academic events. • Open-Source Contributions: If applicable, contribute tools or datasets back to the community.

Applying Archaeological Principles

To effectively borrow from archaeology, consider the following principles:

• Stratigraphy: Just as archaeologists study layers of soil, examine layers of software (e.g., presentation, business logic, data access) to understand the buildup and interaction over time. • Typology: Develop a classification system for architectures, akin to how artifacts are grouped by material, form, or function. • Seriation: Arrange architectures in a sequence based on certain attributes to study chronological development. • Contextualization: Always interpret architectures within the context of their environment, recognizing that external factors significantly influence their form and function.

Defining Software Architecture

Based on this method, you can define software architecture by:

• Structural Elements: Identifying the key components and their configurations. • Relationships and Interactions: Understanding how components interact and depend on each other. • Principles and Guidelines: Recognizing the underlying principles that

guide architectural decisions. • Constraints: Acknowledging limitations imposed by technology, resources, or requirements.

What Software Architecture Is Not

Through this approach, you can delineate that software architecture is not:

- Just Code: It's more than lines of code; it's the high-level structure.
- Single Design Patterns: While design patterns inform architecture, they are not architectures themselves.
- Implementation Details: Architecture focuses on the big picture rather than low-level implementation specifics.

Benefits of This Approach

- Comprehensive Understanding: Gain a holistic view of software architectures across different contexts.
- Historical Insight: Learn how architectures have evolved and why certain practices emerged.
- Best Practices Identification: Isolate architectural strategies that lead to successful software systems.
- Cross-Disciplinary Innovation: Enrich information systems research with methodologies from archaeology.

Final Thoughts

By treating software architectures as artifacts, you open up new avenues for analysis and understanding. This archaeological approach allows you to systematically define software architecture, appreciate its complexities, and distinguish it from related concepts. It fosters a deeper appreciation of the socio-techni

Reasoning Behind the Proposed Method

Overview:

By adopting archaeological research methods, we can systematically analyze software architectures as artifacts that embody historical, cultural, and technical information. This approach allows us to:

- Deconstruct software architectures to understand their components and relationships.
- Contextualize architectures within their socio-technical environments.
- Compare and classify architectures to identify patterns and evolutions.
- Define what constitutes software architecture by distinguishing it from other software elements.

1. Artifact Excavation (Data Collection)

Reasoning:

- Comprehensive Sampling: Like archaeologists excavating artifacts from various sites, collecting a wide range of software architectures ensures a representative sample for analysis.
- Multiple Sources: Gathering data from code repositories, documentation, and expert interviews provides a holistic view, capturing both technical details and human insights.

- Inclusion of Legacy Systems: Studying older architectures helps trace the evolution of architectural practices and understand historical contexts.

2. Classification and Cataloging

Reasoning:

- Organization: Classifying architectures aids in organizing the data, making it manageable and facilitating pattern recognition.
- Multiple Classification Dimensions: By categorizing architectures structurally, functionally, and temporally, we capture different facets that may influence architectural decisions.
- Use of Tools: Employing databases and taxonomy software enhances accuracy and efficiency in cataloging.

3. Structural Analysis

Reasoning:

- Component Breakdown: Identifying fundamental components reveals the building blocks of architectures.
- Relationship Mapping: Understanding interactions between components highlights dependencies and communication pathways.
- Pattern Recognition: Identifying common patterns and anti-patterns helps in understanding best practices and pitfalls.
- Metric Calculation: Quantitative metrics provide objective measures to compare architectures.

4. Contextual Analysis

Reasoning:

- Holistic Understanding: Contextual factors significantly influence architectural decisions; thus, analyzing these factors provides insights beyond technical aspects.
- Historical Context: Recognizing technological limitations and advancements of different eras explains why certain architectural choices were made.
- Organizational and Economic Factors: These factors can constrain or drive architectural innovation, affecting the final design.

5. Comparative Analysis

Reasoning:

- Trend Identification: Comparing architectures helps identify common trends, shifts in practices, and emerging patterns.
- Learning from Differences: Understanding divergences can reveal why certain architectures succeed or fail in specific contexts.
- Best Practices: Isolating features associated with successful projects aids in defining effective architectural principles.

6. Interpretation and Hypothesis Formation

Reasoning:

- **Synthesis of Findings:** Interpreting the data allows for the formulation of a precise definition of software architecture.
- **Distinguishing Boundaries:** Clearly defining what is not part of software architecture prevents ambiguity and scope creep.
- **Theory Development:** Proposing theories based on empirical data strengthens the academic rigor of the research.

7. Documentation and Preservation

Reasoning:

- **Knowledge Retention:** Thorough documentation ensures that findings are preserved for future reference and research continuity.
- **Accessibility:** Storing data in accessible repositories promotes transparency and facilitates peer review.
- **Standardization:** Consistent documentation practices enhance collaboration and interoperability among researchers.

8. Dissemination

Reasoning:

- **Community Contribution:** Sharing findings enriches the collective understanding and spurs further research.
- **Feedback Loop:** Presenting at conferences and publishing allows for critique and refinement of the research.
- **Practical Impact:** Open-source contributions can have immediate benefits for practitioners in the field.

Ensuring Completeness

After reviewing the proposed method, here are additional elements to consider to ensure the research is comprehensive:

A. Incorporating Ethnographic Methods

Reasoning:

- **Human Factors:** Software architecture is not only a technical construct but also a product of human collaboration and decision-making.
- **Cultural Influences:** Team dynamics, organizational culture, and individual expertise can shape architectural outcomes.

Implementation:

- **Participant Observation:** Engage with development teams to observe the architectural decision-making process.
- **Narrative Analysis:** Analyze stories and anecdotes shared by practitioners to gain insights into tacit knowledge.

B. Technological Evolution Analysis

Reasoning:

- Tool Influence: The availability and popularity of certain tools and technologies can influence architectural choices.
- Innovation Adoption: Studying how new technologies are adopted into architectures can reveal patterns of innovation diffusion.

Implementation:

- Technology Lifecycle Mapping: Chart the emergence and decline of technologies within architectures over time.
- Adoption Rate Analysis: Examine how quickly new technologies are incorporated into existing architectures.

C. Socio-Economic Impact Assessment

Reasoning:

- Economic Drivers: Cost considerations, market competition, and resource availability can dictate architectural decisions.
- Societal Needs: Social demands and user expectations can shape the functionality and structure of software systems.

Implementation:

- Cost-Benefit Analysis: Assess how economic factors influence architecture, such as trade-offs between performance and cost.
- User Demand Studies: Analyze how user requirements impact architectural complexity and design.

D. Cross-Disciplinary Integration

Reasoning:

- Enriched Perspectives: Integrating insights from fields like anthropology, sociology, and cognitive science can deepen the understanding of architecture as a socio-technical artifact.
- Holistic Analysis: Cross-disciplinary approaches help in examining the interplay between technology and human factors.

Implementation:

- Interdisciplinary Workshops: Collaborate with experts from other fields to broaden the analytical framework.
- Literature Review: Incorporate theories and models from other disciplines that are relevant to architectural analysis.

E. Addressing Limitations and Biases

Reasoning:

- Research Validity: Acknowledging and mitigating biases enhances the credibility of the research.
- Generalizability: Understanding the limitations ensures that conclusions are appropriately framed.

Implementation:

- Bias Identification: Reflect on potential biases in data selection and interpretation.
- Limitations Section: Explicitly state the research limitations in documentation and publications.

Final Thoughts

Summary:

- The proposed method is grounded in adapting archaeological principles to systematically study software architectures.
- Each step is designed to uncover different layers of understanding, from technical structures to socio-cultural contexts.
- The inclusion of additional considerations like ethnographic methods and cross-disciplinary integration enriches the research.

Conclusion:

By thoroughly examining software architectures as artifacts, we can develop a nuanced definition that captures their essence and distinguishes them from other software elements. This comprehensive approach ensures that we consider all relevant factors, leading to robust and insightful findings.

What Software Architecture Is and Is Not

Based on the Method:

- Is:
 - A high-level abstraction of a software system's structure.
 - Defined by its components, their interactions, and the guiding principles behind their arrangement.
 - Influenced by technical, organizational, and socio-economic factors.
- Is Not:
 - Merely the source code or low-level implementation details.
 - A single design pattern or technology stack.
 - Static; it evolves with changing contexts and requirements.

Ensuring Nothing Was Missed

- Comprehensiveness: The method covers data collection, analysis, interpretation, and dissemination.
- Multi-Faceted Analysis: Includes technical, historical, organizational, and socio-economic perspectives.
- Additional Considerations: Ethnographic methods, technological evolution, socio-economic impacts, cross-disciplinary integration, and bias mitigation have been incorporated.

Confidence:

With these additions and the thorough reasoning provided, the research method is comprehensive and well-suited to define what software architecture is and is not by viewing it through the lens of archaeology.

Bibliography

- Aristotle (1933). *Aristotle in 23 Volumes*. Trans. by H. Tredennick. Vol. 17, 18. Cambridge, MA; London: Harvard University Press; William Heinemann Ltd.
- Capps, J. (2023). “The Pragmatic Theory of Truth”. In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta and U. Nodelman. Summer 2023. Metaphysics Research Lab, Stanford University. URL: <https://plato.stanford.edu/archives/sum2023/entries/truth-pragmatic/>.
- David, M. (2022). “The Correspondence Theory of Truth”. In: *The Stanford Encyclopedia of Philosophy*. Ed. by E. N. Zalta. Summer 2022. Metaphysics Research Lab, Stanford University. URL: <https://plato.stanford.edu/archives/sum2022/entries/truth-correspondence/>.
- Frank, U. (2007). “Ein Vorschlag zur Konfiguration von Forschungsmethoden in der Wirtschaftsinformatik”. In: *Wissenschaftstheoretische Fundierung und wissenschaftliche Orientierung der Wirtschaftsinformatik*. Ed. by F. Lehner and S. Zelewski. Berlin: GITO, pp. 155–184.
- Husserl, E. (2002). *Ideen Zu Einer Reinen Phänomenologie Und Phänomenologischen Philosophie*. Berlin, Boston: De Gruyter. DOI: doi:10.1515/9783110916096.
- Ogden, C. K. and I. A. Richards (1923). *The Meaning of Meaning: A Study of the Influence of Language upon Thought and of the Science of Symbolism*. 1st ed. New York: Harcourt, Brace & World, Inc.
- Putnam, H. (1973). “Meaning and Reference”. In: *The Journal of Philosophy* 70.19, p. 699. DOI: 10.2307/2025079.
- Putnam, H. (1975). “The Meaning of ”Meaning””. In: *Language, Mind, and Knowledge*. Minnesota Studies in the Philosophy of Science 7. Minneapolis: University of Minnesota Press, pp. 131–193.
- Williamson, T. (2002). *Knowledge and Its Limits*. Oxford University Press. DOI: 10.1093/019925656X.001.0001.
- Wittgenstein, L. (1953). *Philosophical Investigations*. Ed. by G. E. M. Anscombe. New York, NY, USA: Wiley-Blackwell.
- Wittgenstein, L., G. C. M. Colombo, and B. Russell (1933). *Tractatus Logico-Philosophicus*. Ed. by C. K. Ogden. New York, Harcourt, Brace.